

Checking a support agent’s actions in code: a harness that lifts a cheap model on airline customer service

Varun Saran

September 2026

Summary

I built a harness that sits between a customer-service AI agent and the airline database it works on. Before the agent can book, change, cancel or compensate, the harness checks the action against the airline’s written policy, in code, using the real booking records. If the action breaks a rule, it is not carried out; the agent is told exactly which rule and why, and gets to try again.

I tested it on the airline tasks of τ^2 -bench, where an AI plays the customer and a task only counts as solved if the database ends up exactly right. On 20 held-out tasks, a cheap model (GPT-4o-mini) solved 26.2% of its attempts on its own and 43.8% with the harness. The share of tasks it solved on every one of four tries went from 0.0% to 25.0%, and each solved task cost less (\$0.038 against \$0.057). With only 20 tasks the improvement is large but not yet statistically certain (95% interval [-1.2, +36.2] points, $p = 0.088$).

Two comparisons matter as much as the headline. The same harness with an AI model doing the checking instead of code did much worse: it blocked 227 actions, 17 of them correct ones, and ended up barely ahead of no harness at all. And the cheap model with the harness came out level with a stronger model used on its own (43.8% against 47.5%).

1 Why this project

This is the second version of Aegis. The first wrapped question answering on HotpotQA and checked answers with another AI model. A controlled test showed it mostly just declined to answer: its checker rejected correct answers about as often as wrong ones, so every error it caught cost several right answers.

The research on this is fairly clear. A harness pays off when its checker is exact, when a program can say for certain that something is wrong and exactly what. Then the harness can make the model repair its work instead of giving up. Company policy for a support agent is a good fit: most rules (“basic economy flights cannot be changed”, “a cancellation needs one of these four conditions”) can be written as code against the booking database.

2 What I built

The agent is τ^2 -bench’s own reference agent, with the same prompt and the same model. The harness adds one step. Every tool call that would change the database goes through a checker first:

- **The checker** is the airline policy written as Python, from the policy document every agent is given. It reads the live database, never writes to it, and covers cancellation eligibility, flights already flown, basic-economy restrictions, route and trip-type rules on flight changes, checked-bag arithmetic, passenger counts, payment rules and compensation rules.
- **A blocked call** never reaches the database. The agent gets a tool result that names the problem, for example “Reservation H9ZU1C is not eligible for cancellation: it was booked more than 24 hours ago, no flight was cancelled by the airline, the cabin is economy, and it has no travel insurance.”
- **Repair.** The agent can fix the call or tell the customer no. After two failed repairs it has to answer the customer in words.
- **One change at a time.** If the agent bundles several changes in one turn, only the first goes ahead, so later changes are judged against the database after the first has run.

Three rules depend on what the customer said, which code can only approximate: whether they confirmed the change, whether a cancellation reason is covered by insurance, and whether they asked for compensation. Those are keyword rules and are marked as such in the code.

3 How it was tested

τ^2 -bench’s airline domain has 50 tasks, split by its authors into 30 for training and 20 for testing. I used the 30 training tasks to find and fix problems in the checker, then froze it (the file hashes are in `PROTOCOL.md`) and wrote down the test plan before running the 20 test tasks once. The customer was played by GPT-4.1-mini in every run.

Each test task was run four times in each of four setups:

- A. Plain agent, GPT-4o-mini.
- B. Harness with the code checker, GPT-4o-mini.
- D. Harness with an AI checker: the same model is shown the policy, the conversation, the proposed call and the same database records, and asked whether the call complies.
- E. Plain agent on a stronger model, GPT-4.1-mini.

A task attempt counts as solved only if τ^2 -bench’s own scorer passes it: the final database must match the reference and the agent must have told the customer the required details. Comparisons are paired by task, with 95% bootstrap intervals over tasks and a Wilcoxon signed-rank test.

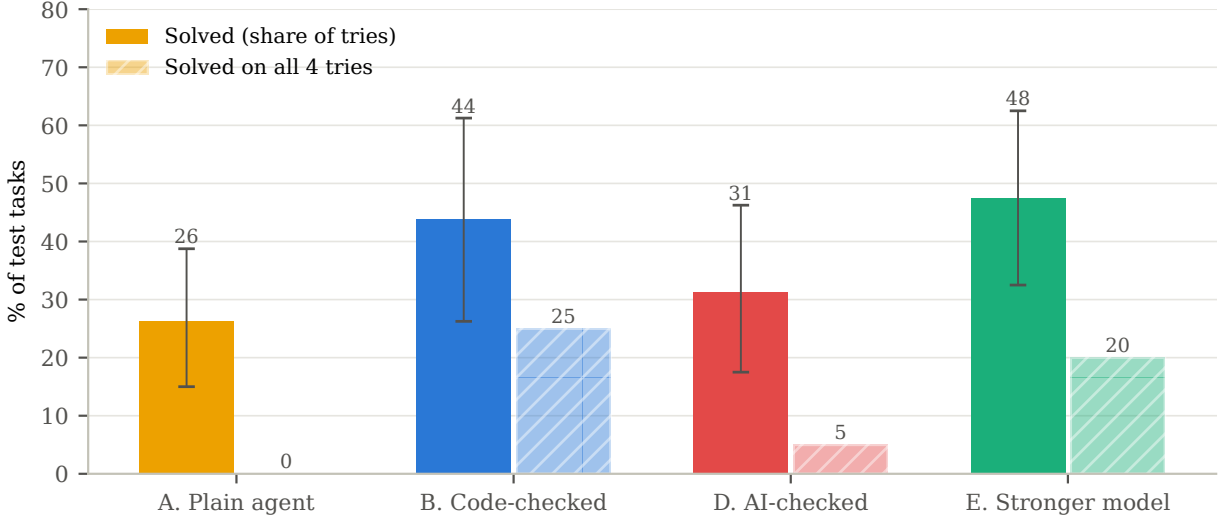


Figure 1: Solid bars: share of attempts solved, with 95% bootstrap intervals over the 20 test tasks. Hatched bars: share of tasks solved on all four tries.

Arm	Agent	Solved (%)	Solved every try	DB correct	\$/conv.	\$/solved	Blocks (false)
A	Plain agent, gpt-4o-mini	26.2 [15.0, 38.8]	0.0%	29.3%	0.0151	0.057	–
B	Harness, code checker, gpt-4o-mini	43.8 [26.2, 61.3]	25.0%	50.0%	0.0166	0.038	72 (3)
D	Harness, AI checker, gpt-4o-mini	31.2 [17.5, 46.2]	5.0%	32.5%	0.0215	0.069	227 (17)
E	Plain stronger model, gpt-4.1-mini	47.5 [32.5, 62.5]	20.0%	50.0%	0.0145	0.031	–

Table 1: Test results. Costs are in US dollars and include the agent, the simulated customer and, for D, the AI checker. “False” blocks are blocked calls identical to the reference answer’s action.

4 Results

The code-checked harness. It raised the solved share from 26.2% to 43.8%, doing better on 10 tasks and worse on 4. The bigger change is in consistency (Figure 2): the plain agent never solved a test task on all four tries, while the harness did so on 25.0% of them. Most of its blocks stopped a real mistake; only 3 of its 72 blocks were on the correct action.

Code checker against AI checker. This is the lesson from the first Aegis, measured directly. Same model, same loop, same database records; the only difference is who judges the call. The AI checker blocked 227 changes, three times as many as the code checker, and 17 of them were the correct action. The agent spent its repairs arguing with a checker that was often wrong, and the harness ended up +5.0 points over no checker at all ($p = 0.696$). The code checker beat it by +12.5 points ($p = 0.083$).

Against a stronger model. The cheap model with the harness solved 43.8% of attempts; GPT-4.1-mini on its own solved 47.5% (a difference of -3.8 points, $p = 0.777$, so no real difference). The harness was more consistent (25.0% against 20.0% solved on every try), and the stronger model was a little cheaper per solved task (\$0.031 against \$0.038). So the harness buys roughly a model tier of reliability, not more.

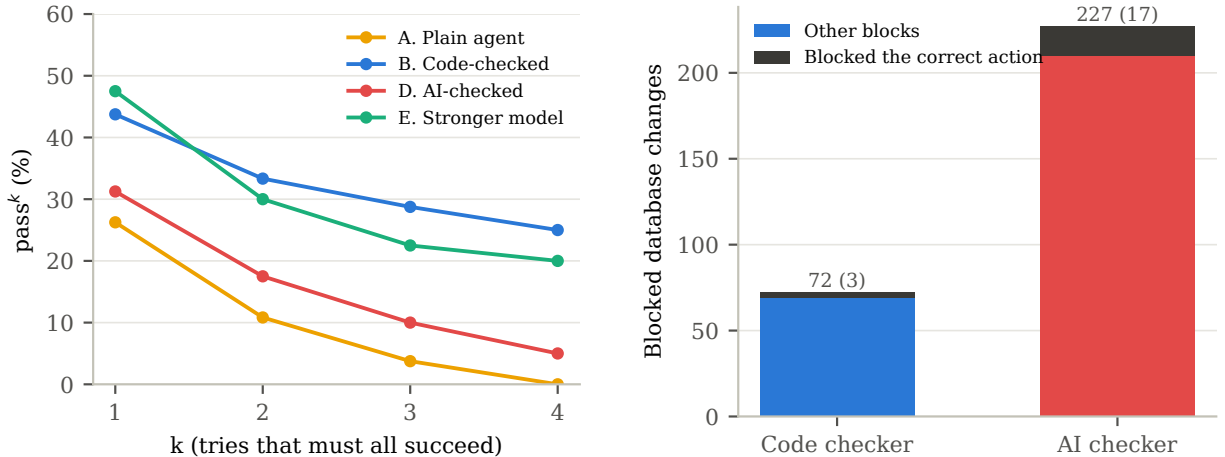


Figure 2: Left: pass^k , the share of tasks solved on all of k tries. Right: database changes each checker blocked, and how many of those were the correct action.

Comparison (paired by task)	Difference	95% CI	Wilcoxon p	Tasks better / worse
Code-checked vs plain	+17.5 pts	[-1.2, +36.2]	0.088	10 / 4
Code checker vs AI checker	+12.5 pts	[-3.8, +27.5]	0.083	10 / 3
Code-checked cheap model vs plain stronger model	-3.8 pts	[-27.5, +20.0]	0.777	7 / 9
AI-checked vs plain	+5.0 pts	[-10.0, +21.2]	0.696	5 / 4
Code-checked vs plain, human-like customer	+10.0 pts	[-7.5, +27.5]	0.406	5 / 3

Table 2: Paired comparisons over the 20 test tasks. The last row used two tries per task instead of four.

Human-like customers. I also ran the plain agent and the harness against a more realistic simulated customer. It writes short, casual messages, gives details only when asked and gets impatient, but keeps every goal and every ID exactly as in the task. The harness still helped (37.5% against 27.5%), but with two tries per task that difference is not reliable ($p = 0.406$).

What the checker caught. Table 3 shows real test conversations, picked automatically. In the first three the plain agent failed the task on every try and the harness solved it; in each, the harness stopped a cancellation the policy does not allow. The last row is a task where the harness did worse than the plain agent.

5 What changed during development

The first version of the checker was too strict in ways the policy is not. It blocked any turn with more than one tool call, and it required a database lookup of the customer where the policy only asks for their user id. In a first trial on three training tasks most of its blocks came from those two rules, so I relaxed both to match the policy text.

Checking every block against the reference answers on the training tasks showed where the rest of the false blocks came from. The exact rules (routes, bag arithmetic, flown segments) never blocked a correct action. The false blocks came from the keyword rule for confirmation, which missed direct instructions such as “Just cancel both reservations”, and from a timing bug: when the agent

Task	Customer’s opening message	Blocked call	What the checker told the agent	Result
24	Hi! I need to remove a passenger named Ethan from my reservation with the code H9ZU1C. Can you help me with that?	<code>cancel_reservation</code>	Reservation H9ZU1C is not eligible for cancellation: it was booked 2024-05-01 04:50 (more than 24 hours ago), no flight was cancelled by the airline, the cabin is economy, and it has no travel insurance. Deny the request.	Solved
44	Hi, I need to review all my upcoming flight reservations. I want to cancel any flights that are longer than 4 hours. Fo...	<code>cancel_reservation</code>	Reservation S61CZX can only be cancelled under its travel insurance, which covers health or weather reasons. Confirm the customer’s reason; a change of plans is not covered, so the request must be denied.	Solved
45	Hi, this is Sophia Taylor. I need to cancel my flight immediately due to a family emergency. My reservation number is P...	<code>cancel_reservation</code>	Reservation PEP4E0 is not eligible for cancellation: it was booked 2024-05-05 05:10 (more than 24 hours ago), no flight was cancelled by the airline, the cabin is <code>basic_economy</code> , and it has no travel insurance. Deny the request.	Solved
8	Hi! I’d like to book a one-way flight from ORD to PHL on May 26. I want to book the exact same flight I took recently o...	<code>book_reservation</code>	Payment method certificate_5094406 is not in the user’s profile; only saved methods can be used.	Failed

Table 3: Real test conversations. Top: tasks the plain agent never solved and the harness did, with the first block in that conversation. Bottom: a task where the harness did worse than the plain agent.

upgraded a booking and cancelled it in the same turn, the checker judged the cancellation against the booking before the upgrade. I fixed both and stopped there.

Development set (30 tasks $\times$ 2 tries)	Solved (%)	Solved both tries (%)	False blocks
Plain agent	25.0	23.3	–
Harness, first version	46.7	40.0	15
Harness, final version	58.3	50.0	8

Table 4: Training tasks only. False blocks are blocked changes identical to the reference action.

I left one known disagreement in place. One training task’s reference answer cancels an insured economy booking because the customer’s plans changed. The policy says insurance covers only health and weather, so the checker blocks it. Following the written policy costs points on that task, and I preferred that to tuning the checker toward the benchmark’s answers.

6 Limitations

Twenty tasks. The test split is small. The harness’s gain over the plain agent is large but its 95% interval $([-1.2, +36.2]$ points) still reaches just below zero. A larger domain, such as τ^2 -bench retail with more than 100 tasks, would settle it.

One domain and a simulated customer. All results are on airline tasks, with another AI playing the customer. Simulated customers make their own mistakes, and a real deployment would face messier requests.

Keyword rules. Three rules read the customer’s words with keyword patterns. On the training tasks they caused most of the false blocks that remained after the fixes; I did not break down the test-set false blocks by rule.

What the checker can’t fix. In the training runs some failures had nothing to do with policy: the agent made the right change but never told the customer a required figure, such as a price difference. A checker on database changes can’t catch that.

7 Cost

Development cost \$2.67 and the test runs \$6.45, \$9.13 in total, including the simulated customers and the AI checker.

8 Reproducing

The code is in the `cs_harness/` folder of

<https://github.com/varunvsaran-wq/Aegis>

Its `README.md` and `PROTOCOL.md` list the exact commands. With an OpenRouter key in `.env`, one test arm is:

```
cd third_party/tau2-bench
.venv/Scripts/python.exe ../../cs_harness/run.py run \
  --domain airline --task-split-name test \
  --agent guarded_code_agent --num-trials 4 \
  --agent-llm openrouter/openai/gpt-4o-mini \
  --user-llm openrouter/openai/gpt-4.1-mini \
  --save-to test_B_code_4omini
```

`cs_harness/analyze.py` computes the statistics and `cs_harness/build_report.py` regenerates every number, table and figure in this document.